

AprilCube: 3D-Printable Fiducial Targets for Reliable 6-DoF Pose Estimation

Younghyo Park and Pulkit Agrawal
MIT
Cambridge, MA

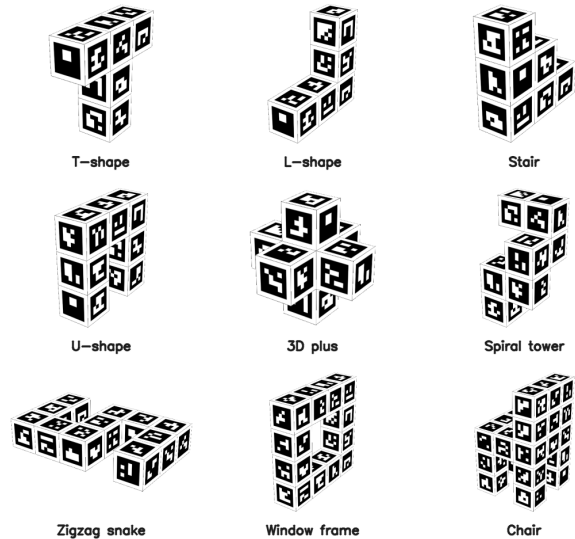
Abstract—AprilCube is an open-source toolkit for generating 3D-printable fiducial targets and estimating their six degree-of-freedom pose from camera images. The repository couples a multi-material 3MF generator with a detector that uses the same configuration file to map observed marker corners to known object-frame geometry and recover the 6-DoF pose of the target object. It represents targets as occupied voxel shapes, with cuboids as a convenient parameterized case and more general voxel-composed geometries for application-specific target designs. This paper describes the software artifact, its geometry model, generated print and simulation assets, pose-estimation pipeline, and streaming API. The project is intended as a reproducible physical tracking target for robotics, calibration, sim-to-real experiments.

Index Terms—fiducial marker, pose estimation, 3D printing, robotics, calibration

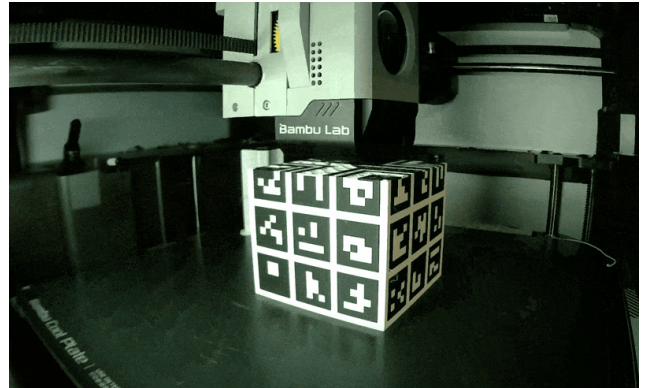
I. INTRODUCTION

Estimating the six degree-of-freedom pose of an object is useful for debugging robot policies, checking manipulation outcomes, aligning simulation and real-world experiments, and building repeatable vision tests. Square fiducial systems such as ArUco and AprilTag are a common low-cost tool for this because the tags are easy to print, encode identifiers, and can be localized by off-the-shelf detectors [1], [2]. A single planar marker or board, however, exposes only one plane of three-dimensional structure. It can leave the camera view under rotation, become ambiguous when viewed from near-frontal single-marker configurations, and lose many correspondences when the target is partially occluded. To get multi-sided visibility, users sometimes make small tracking props by printing tags on paper and attaching them to the faces of a cube or box. This can work, but it is mechanically fragile and geometrically unforgiving: paper labels can peel, wear, or warp, and each tag must be attached with the correct scale, rotation, and position. Even small misalignments become errors in the assumed marker geometry and can bias the recovered object pose.

AprilCube addresses these practical and geometric limits by making the marker pattern part of a printable three-dimensional object. Instead of manually aligning, rotating, and adhering separate paper tags, the user generates a single target whose marker layout and physical geometry are defined together. Modern desktop 3D printers operate at sub-millimeter scales, so tag placement is transferred from a human assembly step to a repeatable fabrication process. The printed target therefore removes a major source of pose error: the detector can use the designed marker geometry directly, rather than



(a) Voxel-composed target gallery



(b) 3D printing still

Fig. 1. AprilCube couples generated fiducial target models with printable physical artifacts. (a) Voxel-composed example targets generated from YAML specifications; each exposed voxel face carries one marker. (b) A picture of the 3D printing process.

assuming that hand-attached labels landed exactly where intended. Markers are placed on exposed surfaces of a known target, and every marker corner is represented in a common object coordinate frame. As the object rotates, different surfaces become visible, but the detector can aggregate all visible corners into a single Perspective-n-Point (PnP) problem. The target therefore behaves less like a flat calibration sheet and

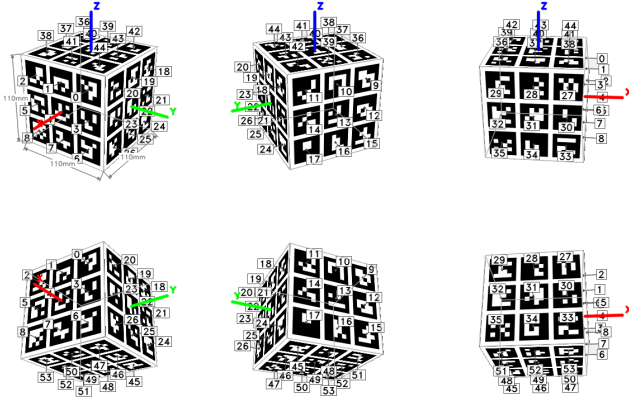


Fig. 2. Generated model preview. The generator renders marker identifiers and object-frame axes before fabrication, and the detector configuration records the marker-corner coordinates explicitly.

more like a small tracked object that can be grasped, moved, placed in a robot workspace, or loaded into a simulator.

The repository contains two tightly coupled components. The generator produces printable targets, model previews, and simulation assets. The detector consumes the generated configuration and estimates the target pose from images or video. This shared data model is the central design choice: the printed artifact, the detector, and the simulation mesh are all derived from the same marker dictionary, physical cell size, margins, borders, tag identifiers, and target geometry. Regular cuboids can be reconstructed analytically from the grid, while more general occupied voxel shapes store explicit marker-corner coordinates.

The generator treats every target as a set of occupied cells with markers on exposed faces. A cuboid grid is the compact default interface for boxes and flat rectangular props, while YAML specifications can describe the same idea as unions of axis-aligned voxel blocks or as explicit occupied voxel grids. This unified view covers cube-like tracking props as well as shape-rich targets whose silhouette, graspability, or visibility profile can be chosen for a particular robotics experiment.

The main contributions are:

- a generator for ArUco/AprilTag voxel-shaped targets with configurable cuboid grids, tag sizes, margins, borders, marker identifiers, and YAML geometry specifications;
- an asset pipeline that emits multi-color 3MF files, detector JSON, preview images, textured OBJ assets, and MuJoCo MJCF models from the same geometry;
- a Python detector that reconstructs exact 3D marker-corner positions from regular cuboid grids or explicit per-marker records, aggregates multi-face observations, solves robust PnP, filters video poses, and exposes synchronous, asynchronous, and visualization-oriented APIs.

II. RELATED WORK

AprilCube builds on square fiducial systems such as ArUco and AprilTag, which provide marker dictionaries, image-space detection, and pose estimation from known marker geometry

[1], [2]. OpenCV’s ArUco module already supports multi-marker boards whose marker-corner coordinates are specified in a common reference frame; these boards are not restricted to a single plane and can represent general 2D or 3D layouts [3]. Marker-mapping work similarly treats fixed markers as one reference system, estimating marker locations from image sequences and then using the map for localization [4]. AprilCube is therefore not a new marker code or a replacement for board and marker-map abstractions. Its contribution is an artifact pipeline that derives the printable model, detector configuration, preview, OBJ mesh, and MJCF model from the same target specification.

Several projects have studied non-coplanar marker placement directly. Oščádal et al. proposed a 3D ArUco grid board with tags placed on angled planes and a library for combining their poses, reporting improved stability compared with a planar grid board [5]. AprilTags3D uses multiple non-coplanar dynamic AprilTags displayed on LCD screens for swarm-robot latching and formation behaviors [6]. These systems support the same geometric intuition used here: distributing fiducial corners over multiple planes improves pose conditioning and visibility. They are, however, focused on particular target designs and application contexts rather than a general generator for printable and simulation-ready target shapes.

The closest project in scope is Fiducial Objects, which designs and evaluates three-dimensional fiducial objects with custom-shaped markers on polygonal faces, along with tools for object configuration estimation from images [7]. AprilCube takes a complementary path. It uses standard ArUco and AprilTag dictionaries for compatibility with OpenCV, assumes the generated CAD geometry is the authoritative source of marker-corner positions, and emits **Bambu Studio** 3MF plus simulation assets for regular cuboid grids and more general voxel-composed shapes. Similar projects therefore exist, but AprilCube is positioned as a reproducible fabrication and software artifact rather than a new fiducial family or a general theory of marker maps.

III. DESIGN GOALS

AprilCube was designed around the practical workflow of making a physical target, printing it, and then using it in camera-driven robotics software without manually transcribing geometry. The first goal is self-description: every printed object has a configuration file that fully specifies the dictionary, tag identifiers, dimensions, and face layouts. The second goal is consistency across media: the 3MF, preview image, OBJ mesh, MJCF model, and detector all use the same object coordinate frame. The third goal is usable video tracking under imperfect observations: the detector should use whatever faces are visible, reject inconsistent marker detections, and rely on temporal filtering or prediction only for very short gaps when the current frame has too few reliable corners.

In practice, each generated target is a matched bundle rather than a set of independent files. The same JSON file that describes the printed target is used by the detector and by the simulation assets, so users do not need to re-enter marker

positions or maintain separate geometry descriptions. This is especially useful for small projects where tracking hardware, motion-capture systems, or carefully manufactured calibration rigs are too expensive or cumbersome, but where a planar printed marker is too fragile a pose reference.

The design also favors inspectability. Generated thumbnails display the marker identifiers, face geometry, and object axes before fabrication. The detector reports visible faces, tag ids, inlier counts, reprojection error, and a debug overlay in addition to the pose. Those diagnostics are intentionally simple because many failures in fiducial tracking are practical failures: the camera is out of focus, the print contrast is poor, the wrong configuration file was loaded, or the marker is viewed at an angle where only a small number of corners remain reliable.

IV. REPOSITORY ARTIFACT

AprilCube is packaged as a Python project with a command-line entry point, a public Python API, generated model assets, examples, and synthetic tests. The core runtime dependencies are NumPy and OpenCV’s contributed ArUco module [8]. Optional visualization uses viser and the textured OBJ model exported by the generator.

The repository includes 37 generated model directories: 28 regular cuboid-grid examples and nine more expressive voxel-shape examples. The cuboid-grid examples cover one to ninety total tags, with tag sizes from 12 mm to 30 mm and side lengths from 30 mm to 180 mm. The more expressive examples include T-, L-, U-, stair-, plus-, spiral-, serpentine-, frame-, and chair-like shapes. Each model directory stores a multi-color 3MF file, a detector configuration file, a rendered preview, and MuJoCo-compatible assets. This model zoo is useful in two ways: users can print a target without first learning the generator options, and the checked-in outputs serve as examples of the exact file bundle produced by the code.

The public package interface is intentionally small. Users can call `aprilcube.detector()` with either a model directory or a configuration file, plus camera intrinsics as a JSON file, dictionary, or 3-by-3 matrix. The returned estimator processes OpenCV frames and reports pose, reprojection error, visible faces, detected tags, and optional visualization overlays. The command-line interface exposes generation and webcam-oriented workflows for users who do not need to write Python code.

V. VOXEL TARGET GENERATION

The generator represents targets as occupied cells with markers on exposed faces. The simplest interface is a cuboid grid with marker dictionary, tag identifiers, a size $g_x \times g_y \times g_z$, and either tag size or cell size. The grid describes how many markers lie along the object axes rather than how many markers lie on one particular face. This makes rectangular cuboids natural: a $5 \times 4 \times 1$ target, for example, becomes a flat box with large top and bottom faces and narrow side strips, while a $3 \times 3 \times 3$ target becomes a conventional cube.

TABLE I
MAIN REPOSITORY ARTIFACTS

Artifact	What it contributes
<code>generate.py</code>	Cuboid-grid geometry, YAML voxel-shape parsing, marker textures, 3MF writing, thumbnail rendering, OBJ and MJCF export.
<code>detect.py</code>	Marker detection, 2D–3D aggregation, PnP pose solve, temporal filtering, overlays, async processing, and viser integration.
<code>models/*_cube/</code>	Pre-generated printable and simulation-ready cubes and cuboids.
<code>models/*_target/</code>	More expressive voxel-shape targets with explicit per-marker geometry records.
<code>test_detection.py</code>	Synthetic renderer, discrete viewpoint checks, trajectories, video output, and image degradations.
<code>examples/</code>	Webcam, web-based visualization, and YAML target specifications.

A marker with dictionary bit width d occupies

$$m = d + 2 \quad (1)$$

cells after including its one-cell black border. With border b , inter-tag margin s , and n tags along one object axis, the total number of cells along that axis is

$$a(n) = 2b + nm + \max(0, n - 1)s. \quad (2)$$

The physical dimensions are then $c[a(g_x), a(g_y), a(g_z)]$, where c is the cell size in millimeters. The total number of required markers is

$$N = 2(g_x g_y + g_x g_z + g_y g_z). \quad (3)$$

The six faces are assigned stable outward normals and face-local “right” and “down” directions. The convention is chosen so that the cross product of the face-local right and down axes equals the outward normal. This convention is used in three places: texture layout, mesh triangle winding, and detector-side corner reconstruction. Because all three are derived from the same face definitions, the rendered preview, the exported simulation mesh, and the detector agree on the object frame.

More expressive targets use the same rule with an occupancy set $V \subset \mathbb{Z}^3$. In the YAML interface, V can be specified as a union of axis-aligned integer cuboids or as an explicit voxel grid. A voxel face receives a marker when its neighboring voxel in the outward normal direction is empty. The number of markers is therefore

$$N_V = \sum_{p \in V} \sum_{f \in \{\pm x, \pm y, \pm z\}} \mathbf{1}[p + n_f \notin V], \quad (4)$$

where n_f is the unit neighbor offset for face direction f . The bounding dimensions are the occupied voxel extent multiplied by the voxel size. This simple rule is sufficient to create the examples in Fig. 1(a), including targets with holes, concavities, overhang-like silhouettes, and multiple disconnected-looking exterior patches that are still face-connected through the voxel body.

The YAML schema separates geometry, marker, size, layout, and material settings. A `voxel_cuboids` target stores a list of integer cuboids with `origin` and `size`; a `voxel_grid` target can enumerate occupied voxels directly. Both paths share dictionary selection, identifier ranges, tag or cell size, border cells, material inversion, and output location. This keeps regular cuboids scriptable from command-line flags while allowing more expressive targets to be checked into the repository as readable specifications.

The generator writes a configuration JSON that maps every marker identifier to physical geometry needed by the detector. For regular cuboids, this includes the dictionary name, grid, tag identifiers, per-face id layout, cell size, tag size, margins, borders, and box dimensions. For general voxel shapes, the JSON additionally stores one record per marker containing its voxel index, face direction, outward normal, face corners, and marker corners in millimeters. The detector does not infer the printed geometry from the image; it reconstructs it exactly from this JSON.

For physical fabrication, the project writes 3MF files with [Bambu Studio](#) paint-color attributes so white cells can be assigned to a second extruder on dual-material printers. The mesh is constructed at cell resolution: black cells are part of the base material, while white cells are marked with the alternate paint color. This keeps the output as a single [Bambu Studio](#) 3MF package rather than separate STL shells.

For simulation and visualization, the same face textures are packed into texture atlases and exported as Wavefront OBJ, MTL, and MJCF files for MuJoCo [9]. Regular cuboid grids use a compact atlas over the six box faces, while general voxel shapes use atlases over their exposed marker faces. The exported OBJ uses meter units and an object frame consistent with detector output. The MJCF file adds a textured visual mesh and collision geometry, making the target convenient for simulated robotics scenes where the visual marker pattern and physical collision shape should be aligned.

VI. FABRICATION WORKFLOW

The intended fabrication workflow begins with either a cuboid grid or a YAML target specification, plus a dictionary choice. Small cubes such as $1 \times 1 \times 1$ or $2 \times 2 \times 2$ are compact physical props, while larger grids provide more visible tags and therefore more PnP correspondences. Flat boxes such as $5 \times 4 \times 1$ behave like thick marker boards with side markers that keep the object observable under roll or pitch. More general occupied shapes add another design axis: the user can choose a silhouette or physical affordance, such as a frame, chair, or stair shape, while still retaining exact marker geometry in the detector configuration. The generator checks that the requested dictionary contains enough unique identifiers and then writes the complete output directory.

The 3MF output is intended to be opened in [Bambu Studio](#); other slicers are not a supported target because the material assignment relies on [Bambu Studio](#) paint-color attributes. In a typical dual-color FDM workflow, the base material is black and the painted cells are white. The choice is conventional

rather than mathematical: the detector ultimately sees image contrast. In practice, matte materials, adequate lighting, and a camera exposure that avoids clipping the white cells matter more than the exact brand of filament.

VII. POSE ESTIMATION, TRACKING, AND API

The detector first loads the JSON configuration and reconstructs a map from marker id to its four three-dimensional corner coordinates. Regular cuboid grids are reconstructed from the grid and face definitions; general voxel-shape configurations can provide the corner map directly through the per-marker records written by the generator. The corner order matches OpenCV’s marker convention, so image corners can be paired directly with object-space corners. For every frame, the detector converts the image to grayscale, applies CLAHE contrast normalization, runs OpenCV’s ArUco detector with the configured ArUco or AprilTag dictionary, filters detections to the target’s known id set, and aggregates all observed marker corners into a single 2D–3D correspondence set.

The OpenCV detector parameters are tuned for the visual properties of multi-material 3D prints. The accurate preset uses sub-pixel corner refinement, a wide adaptive-threshold window range, relaxed candidate perimeter limits, and dense perspective sampling. A faster preset reduces threshold passes and skips sub-pixel refinement for webcam use.

Given image points u_i , object points X_i , and camera matrix K , the pose is estimated by minimizing

$$\sum_i \|\pi(K(RX_i + t)) - u_i\|_2^2, \quad (5)$$

where R, t are the object-to-camera rotation and translation and π is perspective projection. With at least six points the implementation uses `solvePnP`; otherwise it uses direct `solvePnP`. Cold starts use `SQPnP` [10], while prior-seeded solves use OpenCV’s iterative method; successful poses are refined with Levenberg–Marquardt on the inlier set. Because visible markers can span multiple non-coplanar faces, the system often has a richer point distribution than a single planar target. General voxel shapes use the same PnP machinery; their only difference is that the object points come from explicit marker records rather than from a regular box layout.

Several implementation details are included for real-time robustness. If the primary detector finds no valid target markers and a prior pose is available, a sharpened frame is passed through a relaxed fallback detector. Detected quads are scored for area, convexity, and side-length balance before entering PnP, and rejected ArUco candidates can be compared against projected marker locations and recovered when their corners are geometrically plausible. After PnP, detections with unusually high per-tag reprojection error can be dropped and the pose re-solved. A face-normal check rejects flipped PnP solutions by verifying that visible face normals point toward the camera, and a temporal jump gate rejects sudden pose discontinuities that are inconsistent with the velocity estimate.

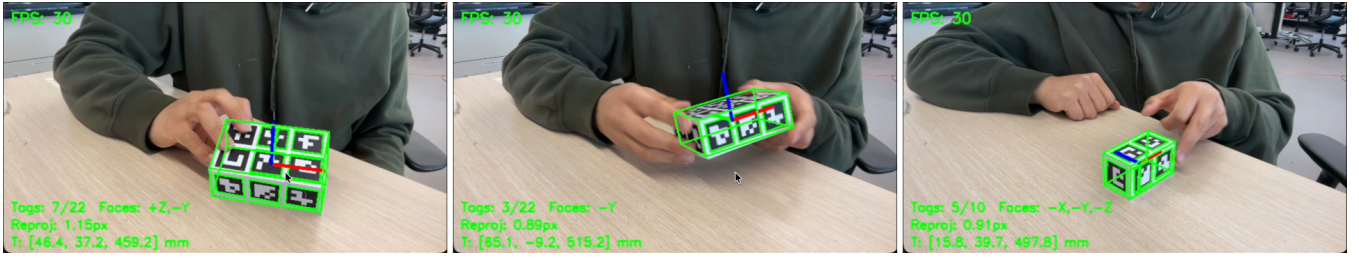


Fig. 3. Real-camera AprilCube tracking stills from the project demo video. The detector overlays marker outlines, a target wireframe, and estimated pose axes while the target is moved through different orientations.

For streaming applications, AprilCube includes an optional error-state Kalman filter. Translation is modeled as position and velocity in millimeters. Rotation uses a nominal unit quaternion with a small-angle error state and angular velocity. This multiplicative formulation avoids filtering Euler angles and avoids treating quaternion components as independent Euclidean coordinates. After each correction, the small-angle rotation error is folded into the nominal quaternion and reset. The process noise adapts to estimated speed: slow motion receives lower acceleration and angular-acceleration noise, while fast motion receives higher noise. Measurement covariance is scaled by reprojection error, tag count, and inlier ratio; a pose with many inlier markers and low reprojection error is trusted more than a pose from a small or noisy correspondence set. Mahalanobis gating down-weights soft outliers and reinitializes the filter after hard outliers or repeated inconsistencies.

The same temporal state is used conservatively outside the filter update. A prior pose seeds PnP when available, and a Kalman prediction can seed reinitialization after a short dropout or provide a predicted pose when detection fails briefly. When no marker detections survive, the detector can track previously observed corners with Lucas–Kanade optical flow; if a predicted pose is available, projected corner locations initialize the search. Optical-flow poses must satisfy a looser reprojection threshold and are passed to the Kalman update with inflated measurement noise so they are trusted less than fresh marker detections.

Figure 3 shows stills from a real-camera tracking demo. The same overlay is produced by the public detector API and displays detected marker outlines, the estimated target wireframe, object-frame axes, visible faces, reprojection error, and translation.

The public estimator supports both synchronous and background use. The main call, `process_frame()`, returns camera-frame pose, reprojection error, inlier count, detected tag ids, visible faces, and a debug overlay. The asynchronous mode is swap-not-queue: submitted frames replace any pending frame, so the worker processes the freshest available image instead of accumulating stale frames. For robotics and visualization, the estimator can convert camera-frame poses into a world frame through an optional extrinsic matrix. The viser integration displays the textured target mesh, object axes, camera feed, and status metadata in a web scene while the

user’s capture loop continues to call `process_frame()`.

VIII. SOFTWARE INTERFACE

The package-level interface is meant to hide the repository layout while preserving access to the underlying estimator. A typical program calls `aprilcube.detector(model, intrinsics)`, where `model` can be either a model directory or a `config.json` path. Intrinsic can be supplied as a calibration JSON file, a dictionary containing focal lengths and principal point, or a NumPy camera matrix. This keeps quick tests concise while still allowing calibrated camera parameters and distortion coefficients in real deployments.

Each call to `process_frame()` returns a dictionary rather than a custom opaque object. The dictionary contains a success flag, Rodrigues rotation vector, translation vector in millimeters, homogeneous camera-frame transform, detected tag ids, visible faces, inlier count, reprojection error, and a rendered debug visualization. This representation is deliberately easy to log, print, serialize, and inspect during experiments. When an extrinsic world transform is provided, `world_pose()` converts the detected object pose into a meter-scale world-frame transform.

The examples in the repository exercise three integration styles. A minimal webcam script shows direct OpenCV capture and frame-by-frame detection. A viser example demonstrates web-based 3D visualization with the generated textured mesh. The asynchronous API supports interactive applications where rendering, robot control, or user-interface work should continue even when marker detection takes longer than one camera period.

IX. INTENDED USE AND LIMITATIONS

The most natural uses are robotic object tracking, calibration props, physical debugging targets, simulation-to-real visualization, and small lab workflows that need a printable six degree-of-freedom reference object with known geometry. AprilCube is particularly useful when the target may be viewed from multiple sides, when a planar board would be awkward to attach to the object being tracked, or when a non-box shape gives better visibility or mounting affordances.

AprilCube is a software and fabrication artifact, not a complete measurement system. Accuracy depends on camera calibration, print quality, marker contrast, focus, exposure, and the mechanical fidelity of the printed faces. In particular,

FDM artifacts such as layer lines, color bleeding, rounded cell corners, glossy surfaces, support scars, and slicer-specific material assignment can affect corner localization.

Natural next steps include physical benchmark datasets, calibrated real-camera error reports, a documented multi-camera interface, mesh-to-voxel import for arbitrary shapes, and additional generated targets optimized for small objects, wide-baseline cameras, or robot end-effector mounting.

ACKNOWLEDGMENT

The author thanks the Improbable AI Lab for its support.

REFERENCES

- [1] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [2] E. Olson, “Apriltag: A robust and flexible visual fiducial system,” in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 3400–3407.
- [3] OpenCV, “Detection of ArUco boards,” https://docs.opencv.org/3.4/db/da9/tutorial_aruco_board_detection.html, accessed: 2026-05-20.
- [4] R. Muñoz-Salinas, M. J. Marín-Jimenez, E. Yeguas-Bolivar, and R. Medina-Carnicer, “Mapping and localization from planar markers,” *Pattern Recognition*, vol. 73, pp. 158–171, 2018.
- [5] P. Ošćádal, D. Heczko, A. Vysocký, J. Mlotek, P. Novák, I. Virgala, M. Sukop, and Z. Bobovsky, “Improved pose estimation of aruco tags using a novel 3d placement strategy,” *Sensors*, vol. 20, no. 17, p. 4825, 2020.
- [6] L. A. Mateos, “Apriltags 3d: dynamic fiducial markers for robust pose estimation in highly reflective environments and indirect communication in swarm robotics,” 2020.
- [7] P. García-Ruiz, F. J. Romero-Ramirez, R. Muñoz-Salinas, M. J. Marín-Jiménez, and R. Medina-Carnicer, “Fiducial objects: Custom design and evaluation,” *Sensors*, vol. 23, no. 24, p. 9649, 2023.
- [8] G. Bradski, “The opencv library,” *Dr. Dobbs’ Journal: Software Tools for the Professional Programmer*, vol. 25, no. 11, pp. 120–123, 2000.
- [9] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 5026–5033.
- [10] G. Terzakis and M. Lourakis, “A consistently fast and globally optimal solution to the perspective-n-point problem,” in *European Conference on Computer Vision*. Springer, 2020, pp. 478–494.